

Milestone 1: Pokemon VGC Strategy RAG System

Eli Fried (730465100) | DATA 790

Overview

I built a question answering system using 55 strategy guides for Pokemon VGC, the competitive doubles format. I plan on integrating this into an app I am creating, so this project was great for testing out different strategies. This system will take in strategy questions, pull the most relevant chunks from the guides, and answer only from those. If the guides do not cover the question then it refuses to answer instead of guessing.

The pipeline is modeled after Lab 4: load the documents, split them with LangChain's splitter, embed them with text-embedding-3-small through the UNC gateway, store them in Chroma, and answer with gpt-4.1-mini. On top of that I added Self-RAG and an input check.

Chunking Strategy:

I compared three chunk sizes the same way as Lab 4, scoring each one on 20 questions I wrote where I already knew which guide had the answer. A question counts as a success if it hit a chunk from the right guide in the top 5 results. The results of the test are in the table below:

Strategy	Size/Overlap	Chunks	Hit Rate	Precision@5	Avg chars retrieved
Small	300 / 30	1,395	95%	73%	982
Medium	600 / 60	646	95%	75%	2,142
Large	1200 / 120	290	95%	69%	4,821

All three sizes found the right guide every time (except for one tricky question that is poorly worded), so I could not compare across hit rate. Looking at Precision@5 all the strategies hovered around 75% which made it difficult to compare. Most of the chunks returned had the relevant information. Instead, I used the tie breaker which was the average characters retrieved since that is a major factor in cost. Since less characters means a lower cost, I chose the small chunking strategy. If the types of questions users ask are similar to my 20 test questions then this will be fine.

Vector Database

I picked the vector store in Lab 5 where I compared three of them on the same guides and the same embeddings. Recall@10 compares each store's results against an exact search so it measures how consistent the index is

Store	Insert (290 vectors)	Median query	Recall@10
Chroma	0.23s	2.1ms	0.970
Pinecone	0.79s	58.9ms	0.995
Weaviate	1.15s	0.56ms	0.995

Every store was accurate enough, and the speed differences don't really matter here since a 2ms search is nothing next to the full question taking almost 5 seconds with the LLM. Due to this I went with Chroma since it runs locally, does not need an account or API key, and is the easiest to use for me. I touched on this in Lab 5, but Pinecone is slower because it makes a network call. If this were a production product then I would consider using Pinecone since it can handle the load of real users far better.

Self-RAG

Basic RAG trusts what the vector store returns, but a Self-RAG adds checks to ensure the quality of the returned chunks is acceptable. The model grades each returned chunk and throws out the ones that do not help. Also, after answering it checks if each claim is supported by the chunks. If either fails, it refuses to answer instead of guessing. I ran it on the 20 test questions and the 5 out of scope questions.

Metric	Result
In-scope questions answered	95% (19/20)
Out-of-scope questions refused	100% (5/5)
Avg judged score	4.58/5
Avg time per question	5.2 s
Avg cost per question	\$0.00039

The out of scope result is great, it refused all of the questions it was supposed to. While the system did miss one of the in scope questions, it still got that base majority correct.

The first failure came from the question "What is the difference between hyper offense and bulky defense?". This question was refused despite % of the chunks coming from the right guides. The

Self-RAG rejected these chunks since they only mentioned defense, not offense. Since this is a comparison question it needed chunks that discussed both concepts to pass the Self-RAG. This reveals that the grading prompt might need to be loosened.

The second failure came from “What is an island pokemon in teambuilding?”. Again, 4/5 chunks came from the correct guide. The grader only kept a single chunk here so the answer was a single line “a Pokemon that can be safely ignored by the opponent.”. This is correct, but the judge marked it wrong saying that it was “a Pokemon isolated in its role or type”. The judge was trying to score the answer from its own knowledge here and got it wrong. Again probably need to loosen the grader here.

Security

Every question gets checked before any model is called. Empty questions, questions over 500 chars, and phrases that show up in injection attacks get blocked. All six of my test inputs were handled correctly, and none of the real 25 questions were blocked by this. Its a simple phrase list, so a reworded attack could get past. However, this is a good basic system to have in place.

Performance and Cost

Indexing all 1,395 chunks cost about \$0.0013 and that is a single time cost. Each question costs about \$0.00039 and takes about 4.8 seconds. Most of that is the model calls. Self-RAG makes 7 calls per question (5 chunk grades, 1 answer, 1 check) which is why it is so much slower. Scaled up that would be \$0.39 at 1,000 questions, \$3.95 at 10k, and \$39 at 100k. There are some ways to bring this down, although I think this cost is already pretty good:

- Grade all 5 chunks in a single call instead of separately
- Cache answers like we talked about in class
- Keep the chunks as small as possible

What Next

Since I do plan on implementing this in a real app, I would like to evaluate the system on a greater variety of questions. I used pretty easy ones for testing purposes in this project, but real users would likely try much harder ones on occasion.

Another thing I would improve is to give the judge the passages so it can better evaluate the answers produced. This was highlighted by the second failure which gave the correct answer, but it was rejected since the judge did not have the context to determine its correctness.